

Written Assignment #2 Solutions

1 Fibonacci Numbers

1.1 Find a 2×2 matrix A , such that $A \times v_i = v_{i+1}$ for all i .

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

1.2 Give a parallel algorithm to compute $F_1, F_2, \dots, F_n$. Your algorithm must take $O(n)$ work and $O(\log n)$ span. Please prove the cost of your algorithm.

Since matrix multiplication is a monoid, we can use parallel scan to compute $A^0, A^1, A^2, \dots$ in $O(n)$ work and $O(\log n)$ span.

Then we can compute $v_i = v_0 \times A^i$ and get F_i from v_i .

2 Parallel Merging

2.1 Idea 1: Blocking + Sequential Merge

- Write down the work and the span of this algorithm, respectively, using parameter k .

Let $n = |A| + |B|$.

Work: $W(n) = O(\frac{n}{k} \log n + n)$

Span: $D(n) = O(\log n + k)$

- Choose the value of parameter k to make the algorithm work-efficient (i.e., $O(n)$ work) and bound the span in $O(\log(|A| + |B|))$. Show how you find the value of k and prove the cost of the algorithm.

If we set $k = \log n$, we have $W(n) = O(n)$ and $D(n) = O(\log n)$.

2.2 Idea 2: Blocking + Recursive Merge

- Write down the recurrence to compute the work and the span of this algorithm, respectively.

Let $n = |A| + |B|$.

Total work: $W(n) = O(\frac{n}{k} \log n) + \frac{n}{k} \times W(k)$
Total span: $D(n) = O(\log n) + D(k)$

- **Prove that if we choose $k = \Theta(\sqrt{n})$, the span is $O(\log(|A| + |B|))$.**
Now $D(n) = O(\log n) + D(\sqrt{n})$. So we have

$$D(n) = \log n + \log n^{1/2} + \log n^{1/4} + \dots + \log 1 = \log n \times (1 + \frac{1}{2} + \frac{1}{4} + \dots) \leq 2 \log n = O(\log n)$$

3 Friend Recommendation

Let $F_u = \text{GET_FRIENDS}(u)$ and let $d_u = \text{NUM_FRIENDS}(u)$. We use two auxiliary boolean arrays indexed by user id:

- $M[v]$: whether v is a direct friend of u .
- $C[v]$: whether v is a candidate 2-hop friend of u .

Initially all entries are false.

Algorithm 1: Friend Recommendation for user u

```

1  $F_u \leftarrow \text{GET\_FRIENDS}(u)$ 
2 ParallelForEach  $v \in F_u$  do
3    $M[v] \leftarrow \text{true}$ 
4 ParallelForEach  $v \in F_u$  do
5    $F_v \leftarrow \text{GET\_FRIENDS}(v)$ 
6   foreach  $w \in F_v$  do if  $w \neq u$  and  $M[w] = \text{false}$  then
7      $C[w] \leftarrow \text{true}$ 
8
9  $R \leftarrow \text{PACK}(\{v \mid C[v] = \text{true}\})$ 
10 return  $(R, |R|)$ 
```

The algorithm is correct because a user w is inserted into R iff there exists a friend v of u such that $w \in \text{GET_FRIENDS}(v)$, so w is a 2-hop neighbor of u . The condition $w \neq u$ removes u itself, and $M[w] = \text{false}$ removes all direct friends of u . Thus R contains exactly all friends' friends of u that are neither u nor already adjacent to u .

For the cost, marking M takes $O(d_u)$ work and $O(1)$ span. The second parallel loop examines every edge incident to a vertex in F_u , so the work is

$$O\left(\sum_{v \in F_u} \deg(v)\right) \leq O(2m) = O(m).$$

All writes are independent boolean writes, so this loop has $O(1)$ span besides the parallel loop overhead. Finally, PACK can be implemented by a parallel prefix sum, which takes linear work in the array size and $O(\log n)$ span. Therefore the total work is $O(m)$ and the total span is $O(\log n)$.

4 With High Probability and Hash Table

Suppose the number of retries is i . This means that the collision happens for i times. So we have

$$\Pr[t = i] \leq \frac{1}{2^i}$$

For any $x \geq 1$, the probability that we have at least x retries is

$$\Pr[t \geq x] = \sum_{i=x}^{+\infty} \Pr[t = i] \leq \sum_{i=x}^{+\infty} \frac{1}{2^i} = \frac{1}{2^{x-1}}.$$

Take $x = 2k \log n$, we have

$$\Pr[t \geq 2k \log n] \leq \frac{1}{2^{2k \log n - 1}} \leq \frac{1}{2^{k \log n}} = \frac{1}{n^k}.$$

So $t = O(2 \log n) = O(\log n)$ w.h.p.